Leitura — Persistência, testes, OpenAPI, validação e arquitetura: Ktor × Quarkus (Sprint 1, parte 2)

Guia de apoio para a segunda parte da Sprint 1 (vídeo e aulas de 05 e 07/10). Na aula de 14/09, a API de tarefas guardava tudo numa lista em memória. Aqui ela passa a usar PostgreSQL, com o esquema criado por migrações, testes que sobem um banco de verdade num container, a documentação OpenAPI gerada a partir do código, validação com erros em *problem details* e um teste que verifica a arquitetura. Os dois stacks seguem lado a lado: Kotlin/Ktor com Exposed e Java/Quarkus com Hibernate ORM e Panache.

O código deste guia é o dos exemplos `exemplos/ktor-tarefas` e `exemplos/quarkus-tarefas` do repositório `web2-2026-2`, passos 6 a 12 de cada `PASSOS.md`. Os capítulos 1 a 8 mostram o código como ele fica até o Passo 9, num pacote só; os capítulos 10 e 11 mostram o que muda nos passos 11 e 12. Versões: Kotlin 2.4.10, Ktor 3.5.2, Exposed 1.5.0, HikariCP 7.1.0, driver PostgreSQL 42.7.13, Flyway 13.7.0, Testcontainers 2.0.5 (Ktor) e 1.21.4 (Quarkus), Quarkus 3.28.2, PostgreSQL 17 e Java 25. Mensagens de erro, tempos e respostas citados foram conferidos rodando os exemplos nessas versões, com Docker Engine 29.

Como ler: os capítulos seguem a ordem da aula. Quem vai usar só um stack pode pular o capítulo do outro (4 para Ktor, 5 para Quarkus); os capítulos 3, 7, 8, 10 e 11 valem para os dois.

Onde rodar: os exemplos pedem Java 25 e Docker. Quem não tiver os dois na máquina, ou estiver no laboratório, usa um Codespace do repositório (Code → Codespaces): ele já traz Java, Maven, Go, Docker e as dependências baixadas, e os comandos são os mesmos. A seção 12.3 tem os detalhes.

Capítulos:

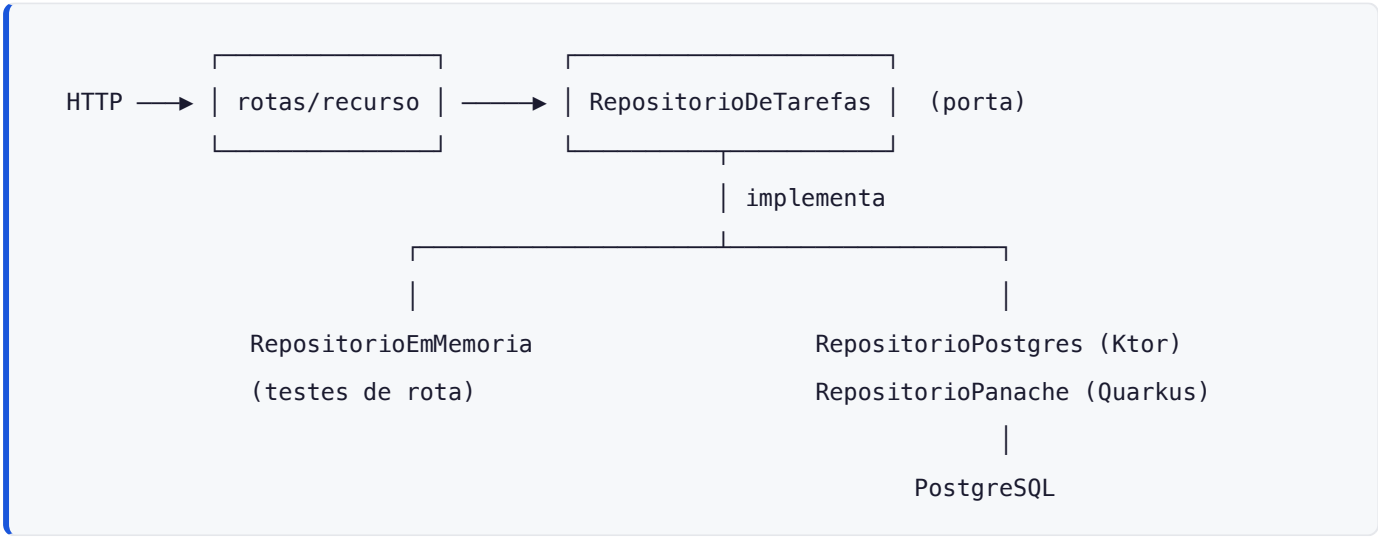
1. Do repositório em memória ao banco
2. JDBC, `DataSource` e pool de conexões
3. Migrações com Flyway
4. Ktor: Exposed
5. Quarkus: Hibernate ORM com Panache
6. Exposed × Panache, lado a lado
7. Testes: da rota ao banco de verdade
8. OpenAPI: a documentação que sai do código
9. Rodar tudo junto
10. Validação e erros em *problem details*
11. Camadas, teste de arquitetura e o CRUD completo
12. Exercícios e dúvidas frequentes

---

1. Do repositório em memória ao banco

1.1 O que muda e o que não muda

Em 14/09, as rotas (Ktor) e o recurso (Quarkus) dependiam de uma interface, `RepositorioDeTarefas`, e a implementação era `RepositorioEmMemoria`. Essa separação mostra seu valor agora: a troca para o banco é uma classe nova que implementa a mesma interface. As rotas mudam só porque ganharam `GET /tarefas/{id}` e a descrição para o OpenAPI; a lógica de acesso a dados fica toda no adaptador.



O `RepositorioEmMemoria` continua no projeto: é o que os testes de rota usam para rodar sem Docker (capítulo 7).

1.2 Por que um banco relacional

Três problemas da lista em memória desaparecem com o banco:

- Os dados somem quando o processo termina. Com PostgreSQL, uma tarefa criada continua lá depois de reiniciar a aplicação (conferido nos dois exemplos).
- Duas instâncias da API não enxergam os dados uma da outra. Com o banco, qualquer instância lê o mesmo estado.
- A lista não era segura para concorrência. Em 14/09, 2.000 `POST` simultâneos perderam tarefas e repetiram ids. Com PostgreSQL, o mesmo teste (2.000 `POST`, 64 em paralelo) deu 2.000 respostas `201`, 2.000 tarefas gravadas e ids únicos, em 7 segundos. Quem garante isso é o banco: `SERIAL` gera ids sem colisão e cada `INSERT` é atômico.

1.3 A tabela

O mesmo esquema para os dois stacks, numa migração (capítulo 3):

```sql
CREATE TABLE tarefas (
    id     SERIAL PRIMARY KEY,
    titulo VARCHAR(200) NOT NULL,
    feita  BOOLEAN NOT NULL DEFAULT FALSE
);

INSERT INTO tarefas (titulo) VALUES ('Estudar Ktor'), ('Estudar Quarkus');
```

- `SERIAL` é um inteiro com uma sequência associada: cada `INSERT` sem `id` recebe o próximo valor.
- `NOT NULL` e `DEFAULT` são regras do banco. Valem mesmo que a aplicação esqueça de validar (seção 5.6 mostra o que acontece).

📖 Ref. PostgreSQL — tipos numéricos e `SERIAL`: https://www.postgresql.org/docs/17/datatype-numeric.html

---

2. JDBC, `DataSource` e pool de conexões

2.1 JDBC em uma página

JDBC é a API padrão do Java para falar com bancos relacionais. Cada banco fornece um driver (para PostgreSQL, `org.postgresql:postgresql`) e a aplicação se conecta por uma URL:

```
jdbc:postgresql://localhost:5432/tarefas
     └── driver ─┘ └─ host ─┘ └porta┘ └ banco ┘
```

Tanto o Exposed quanto o Hibernate usam JDBC por baixo. Eles geram o SQL e convertem linhas em objetos; o transporte até o banco é sempre o driver JDBC.

2.2 Por que um pool

Abrir uma conexão com o PostgreSQL envolve TCP, autenticação e a criação de um processo no servidor. Fazer isso a cada requisição custa caro. Um pool abre algumas conexões e as empresta: a requisição pega uma, usa e devolve.

No Ktor, o pool é escolha sua. O exemplo usa HikariCP:

```
fun criarDataSource(config: ConfigBanco): HikariDataSource = HikariDataSource(
    HikariConfig().apply {
        jdbcUrl = config.url
        username = config.usuario
        password = config.senha
        maximumPoolSize = 5
    },
)
```

No Quarkus, o pool (Agroal) vem com a extensão do driver. Basta dizer o tipo de banco; o resto é configuração (seção 5.2).

- `maximumPoolSize = 5` limita as conexões abertas. No teste de concorrência da seção 1.2, 5 conexões atenderam 64 requisições simultâneas: as que chegam com o pool ocupado esperam a vez.
- O pool é um recurso: feche-o quando a aplicação parar. No Ktor, o exemplo faz isso com `monitor.subscribe(ApplicationStopped) { dataSource.close() }`.

📖 Ref. HikariCP: https://github.com/brettwooldridge/HikariCP

📖 Ref. Exposed — Working with DataSource: https://www.jetbrains.com/help/exposed/working-with-datasource.html

2.3 Onde fica a senha

URL, usuário e senha não entram no código nem no repositório. Os dois exemplos leem de variáveis de ambiente, com um valor padrão para desenvolvimento:

```
data class ConfigBanco(val url: String, val usuario: String, val senha: String) {
    companion object {
        fun doAmbiente() = ConfigBanco(
            url = System.getenv("DB_URL") ?: "jdbc:postgresql://localhost:5432/tarefas",
            usuario = System.getenv("DB_USER") ?: "tarefas",
            senha = System.getenv("DB_PASSWORD") ?: "tarefas",
        )
    }
}
```

```
%prod.quarkus.datasource.jdbc.url=${DB_URL:jdbc:postgresql://localhost:5432/tarefas}
%prod.quarkus.datasource.username=${DB_USER:tarefas}
%prod.quarkus.datasource.password=${DB_PASSWORD:tarefas}
```

Na Sprint 2, o banco passa a ser um PostgreSQL gerenciado, e essas variáveis vêm de segredos do ambiente de implantação. A rubrica pede exatamente isso: connection string em segredo, nunca no repositório.

---

3. Migrações com Flyway

3.1 O problema

O esquema do banco muda ao longo do projeto: uma coluna nova, um índice, uma tabela. Cada máquina (a sua, a do colega, o CI, o banco de produção) precisa chegar ao mesmo esquema, na mesma ordem, sem ninguém rodar SQL à mão. Migração é um script versionado que leva o banco de uma versão para a próxima.

3.2 Como o Flyway funciona

- Os scripts ficam em `src/main/resources/db/migration`, com o nome `V<versão>__<descrição>.sql` (dois sublinhados). O exemplo tem `V1__cria_tarefas.sql`.
- Na primeira execução, o Flyway cria a tabela `flyway_schema_history` e aplica os scripts em ordem de versão. Cada script aplicado vira uma linha com versão, descrição e checksum.
- Nas execuções seguintes, compara os scripts com o histórico e aplica só os novos.

O log da primeira subida do exemplo, contra um banco vazio:

```
Schema history table "public"."flyway_schema_history" does not exist yet
Successfully validated 1 migration
Creating Schema History table "public"."flyway_schema_history" ...
Current version of schema "public": << Empty Schema >>
Migrating schema "public" to version "1 - cria tarefas"
Successfully applied 1 migration to schema "public", now at version v1
```

E da segunda, com o banco já migrado:

```
Current version of schema "public": 1
Schema "public" is up to date. No migration necessary.
```

É isso que a rubrica chama de migração idempotente: rodar de novo não muda nada.

📖 Ref. Flyway — Versioned migrations: https://documentation.red-gate.com/flyway/flyway-concepts/migrations/versioned-migrations

3.3 Onde a migração roda

- Ktor: o exemplo chama o Flyway na subida, antes de montar as rotas.

```
fun migrar(dataSource: DataSource) {
    Flyway.configure().dataSource(dataSource).load().migrate()
}
```

- Quarkus: a extensão `quarkus-flyway` faz o mesmo com uma propriedade.

```
quarkus.flyway.migrate-at-start=true
```

Migrar na subida é o mais simples e serve bem para o projeto. Em sistemas com várias instâncias, é comum migrar num passo separado do deploy; o Flyway usa um lock no banco para que duas instâncias subindo juntas não apliquem o mesmo script duas vezes.

📖 Ref. Quarkus — Using Flyway: https://quarkus.io/guides/flyway

3.4 O esquema é da migração, não do ORM

Hibernate e Exposed sabem criar tabelas sozinhos. A rubrica proíbe: esquema gerado apenas por migrações, sem `ddl-auto` ou equivalente. O motivo é que a geração automática não tem histórico, não sabe renomear coluna sem perder dados e se comporta diferente em cada ambiente.

- Quarkus: `quarkus.hibernate-orm.schema-management.strategy=none`. O Hibernate só lê e escreve dados; não cria nem altera tabelas.
- Ktor: o `object Tarefas` do Exposed descreve a tabela para montar consultas, mas o exemplo nunca chama `SchemaUtils.create`. A tabela vem do `V1`.

3.5 Regras de convivência

- Nunca edite uma migração que já foi aplicada em algum banco. Crie a próxima (`V2__...`).
- Uma migração por mudança, com nome que diga o quê: `V2__adiciona_prazo.sql`.
- Dados iniciais que fazem parte do sistema (categorias fixas, por exemplo) podem ir numa migração, como as duas tarefas do `V1`. Dados de teste não.

> Erro comum: editar o `V1` depois de aplicado. O Flyway guarda o checksum de cada script e recusa subir quando o arquivo muda:
>
> ```
> FlywayValidateException: Validate failed: Migrations have failed validation
> Migration checksum mismatch for migration version 1
> -> Applied to database : 1556295178
> -> Resolved locally    : 1950589099
> Either revert the changes to the migration, or run repair to update the schema history.
> ```
>
> Desfaça a edição e crie um `V2`. `repair` só atualiza o histórico, não o esquema: use quando souber exatamente o que está fazendo, por exemplo num banco local descartável.

> Erro comum: migração com SQL que falha, como criar uma tabela que já existe. O Flyway para e mostra o script, o estado SQL e a mensagem do banco:
>
> ```
> FlywayMigrateException: Failed to execute script V1_1__errada.sql
> SQL State  : 42P07
> Message    : ERROR: relation "t" already exists
> ```
>
> No PostgreSQL, DDL roda dentro de transação: o script que falhou é desfeito inteiro. Conferido com um `V2` que criava uma tabela nova e depois repetia uma existente: a tabela nova não ficou no banco, o `V2` não entrou no histórico, e depois de corrigido o arquivo o Flyway aplicou o `V2` normalmente.

> Erro comum (Ktor): esquecer `flyway-database-postgresql`. Desde o Flyway 10, o suporte a cada banco é um módulo separado. Só com `flyway-core`, a subida falha com `FlywayException: Unsupported Database: PostgreSQL 17.10`. No Quarkus, a extensão já traz o módulo.

---

4. Ktor: Exposed

4.1 O que é o Exposed

Exposed é a biblioteca de acesso a banco da JetBrains para Kotlin. Oferece duas formas de uso:

- DSL: consultas escritas em Kotlin com a forma do SQL (`selectAll().where { ... }`). É a usada no exemplo.
- DAO: entidades com propriedades, no estilo de um ORM.

A DSL deixa o SQL visível e não esconde quando uma consulta acontece; é um bom começo.

Dependências (além do driver, do pool e do Flyway):

```
implementation("org.jetbrains.exposed:exposed-core:1.5.0")
implementation("org.jetbrains.exposed:exposed-jdbc:1.5.0")
```

A partir do Exposed 1.0, os pacotes começam com `org.jetbrains.exposed.v1` . Exemplos da internet com `org.jetbrains.exposed.sql` são da série 0.x e não compilam sem ajuste.

📖 Ref. Exposed — Get started: https://www.jetbrains.com/help/exposed/get-started-with-exposed.html

4.2 A tabela em Kotlin

```
object Tarefas : Table("tarefas") {
    val id = integer("id").autoIncrement()
    val titulo = varchar("titulo", 200)
    val feita = bool("feita").default(false)
    override val primaryKey = PrimaryKey(id)
}
```

- `object` : uma única instância, que representa a tabela. Kotlin para quem conhece Java: é um singleton sem precisar de `static` nem construtor privado.
- Cada coluna é uma propriedade tipada. `Tarefas.titulo` é uma `Column<String>` ; o compilador impede comparar com um `Int` .
- O nome entre aspas (`"tarefas"` , `"titulo"`) é o do banco. Tem de bater com a migração.

4.3 Consultas

O repositório do exemplo:

```
class RepositorioPostgres(private val db: Database) : RepositorioDeTarefas {

    override suspend fun listar(): List<Tarefa> = suspendTransaction(db) {
        Tarefas.selectAll().orderBy(Tarefas.id to SortOrder.ASC).map { it.paraTarefa() }
    }

    override suspend fun buscar(id: Int): Tarefa? = suspendTransaction(db) {
        Tarefas.selectAll().where { Tarefas.id eq id }.singleOrNull()?.paraTarefa()
    }

    override suspend fun adicionar(nova: NovaTarefa): Tarefa = suspendTransaction(db) {
        val id = Tarefas.insert { it[titulo] = nova.titulo } get Tarefas.id
        Tarefa(id, nova.titulo)
    }

    private fun ResultRow.paraTarefa() =
        Tarefa(this[Tarefas.id], this[Tarefas.titulo], this[Tarefas.feita])
}
```

- `selectAll()` monta um `SELECT`; `.where { Tarefas.id eq id }` acrescenta o `WHERE`. `eq` é uma função infixa: `Tarefas.id eq id` é o mesmo que `Tarefas.id.eq(id)`.
- A consulta só vai ao banco quando o resultado é percorrido (`map`, `singleOrNull`).
- `insert { ... } get Tarefas.id` devolve o id gerado pelo `SERIAL`.
- `ResultRow` é uma linha do resultado. A função de extensão `paraTarefa()` converte linha em objeto de domínio, num lugar só.

📖 Ref. Exposed — CRUD operations (DSL): https://www.jetbrains.com/help/exposed/dsl-crud-operations.html

4.4 Transações

Toda operação do Exposed precisa acontecer dentro de uma transação. É a transação que pega uma conexão do pool, faz `COMMIT` no fim ou `ROLLBACK` se houver exceção, e devolve a conexão.

- `transaction(db) { ... }` é a forma bloqueante.
- `suspendTransaction(db) { ... }` é a forma para código com corrotinas, usada no exemplo.

> Erro comum: consultar fora de uma transação. `Tarefas.selectAll().toList()` solto no código falha com `IllegalStateException: No transaction in context.`

4.5 Por que a porta ganhou `suspend`

Em 14/09 (capítulo 9 daquele guia), a regra era não travar a thread do servidor. JDBC é bloqueante: enquanto espera o banco, a thread fica parada. Por isso a interface do Ktor mudou:

```
interface RepositorioDeTarefas {
    suspend fun listar(): List<Tarefa>
    suspend fun buscar(id: Int): Tarefa?
    suspend fun adicionar(nova: NovaTarefa): Tarefa
}
```

Os handlers de rota do Ktor já são `suspend`, então chamar `repositorio.listar()` não muda nada na rota. A implementação com banco usa `suspendTransaction`; a em memória só ganha a palavra `suspend`.

📖 Ref. Exposed — Transactions: https://www.jetbrains.com/help/exposed/transactions.html

4.6 Montagem da aplicação

```
// Produção: banco de verdade, migrado na subida.
fun Application.modulo(config: ConfigBanco = ConfigBanco.doAmbiente()) {
    val dataSource = criarDataSource(config)
    migrar(dataSource)
    monitor.subscribe(ApplicationStopped) { dataSource.close() }
    configurar(RepositorioPostgres(Database.connect(dataSource)))
}

// O resto da aplicação só conhece a porta: os testes podem passar outro repositório.
fun Application.configurar(repositorio: RepositorioDeTarefas) {
    install(Koin) {
        modules(module { single<RepositorioDeTarefas> { repositorio } })
    }
    install(ContentNegotiation) { json() }
    rotas()
}
```

A ordem importa: pool, migração, conexão do Exposed, e só então as rotas. Se a migração falhar, a aplicação nem sobe, o que é melhor do que subir com o esquema errado.

A divisão entre `modulo` e `configurar` é o que permite testar as rotas sem banco (seção 7.2).

---

5. Quarkus: Hibernate ORM com Panache

5.1 Hibernate, JPA e Panache

- JPA (Jakarta Persistence) é a especificação de ORM do Java: anotações como `@Entity` e `@Id` e a API do `EntityManager`.
- Hibernate ORM é a implementação de JPA usada pelo Quarkus.
- Panache é uma camada do Quarkus sobre o Hibernate que elimina o código repetitivo: métodos prontos como `listAll`, `findById` e `persist`.

Dependências no `pom.xml` (versões vêm do BOM do Quarkus):

```
<dependency>
  <groupId>io.quarkus</groupId>
  <artifactId>quarkus-hibernate-orm-panache</artifactId>
</dependency>
<dependency>
  <groupId>io.quarkus</groupId>
  <artifactId>quarkus-jdbc-postgresql</artifactId>
</dependency>
<dependency>
  <groupId>io.quarkus</groupId>
  <artifactId>quarkus-flyway</artifactId>
</dependency>
```

📖 Ref. Quarkus — Simplified Hibernate ORM with Panache: https://quarkus.io/guides/hibernate-orm-panache

5.2 Configuração

```
quarkus.datasource.db-kind=postgresql
%prod.quarkus.datasource.jdbc.url=${DB_URL:jdbc:postgresql://localhost:5432/tarefas}
%prod.quarkus.datasource.username=${DB_USER:tarefas}
%prod.quarkus.datasource.password=${DB_PASSWORD:tarefas}

quarkus.hibernate-orm.schema-management.strategy=none
quarkus.flyway.migrate-at-start=true
```

O prefixo `%prod.` faz a URL valer só no perfil de produção (o `java -jar`). Em `dev` (`quarkus dev`) e `test`, não há URL configurada, e aí entra o Dev Services.

5.3 Dev Services: o banco que aparece sozinho

Com Docker disponível e sem URL configurada, o Quarkus sobe um PostgreSQL num container ao iniciar em modo dev ou nos testes, liga a aplicação a ele e o descarta no fim. O log do teste do exemplo:

```
Dev Services for PostgreSQL started.
Dev Services for default datasource (postgresql) started - container ID is 54b9a6f4c176
Migrating schema "public" to version "1 - cria tarefas"
quarkus-tarefas 0.1.0 on JVM (powered by Quarkus 3.28.2) started in 54.616s.
```

Os 54 segundos incluem baixar a imagem `postgres:17` (158 MB) na primeira vez. Na execução seguinte, o container subiu em 1,5 segundo e a aplicação em 4,4 segundos.

📖 Ref. Quarkus — Dev Services for Databases: https://quarkus.io/guides/databases-dev-services

5.4 A entidade

```java
@Entity
@Table(name = "tarefas")
public class TarefaEntidade extends PanacheEntityBase {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    public Integer id;

    public String titulo;

    public boolean feita;

    Tarefa paraTarefa() {
        return new Tarefa(id, titulo, feita);
    }
}
```

- `@Entity` marca a classe como mapeada; `@Table` diz o nome da tabela da migração.
- `GenerationType.IDENTITY` diz ao Hibernate que o id é gerado pelo banco (o `SERIAL`).
- Campos públicos são o estilo do Panache: na compilação, ele gera getters e setters e troca os acessos a campo por chamadas a eles.
- A entidade não é o `record Tarefa` do domínio. O recurso REST continua devolvendo `Tarefa`; a entidade fica restrita ao adaptador de persistência. `paraTarefa()` faz a conversão.

Por que não anotar o `record` com `@Entity`? JPA exige construtor sem argumentos e campos mutáveis, e um `record` não tem nenhum dos dois.

5.5 O repositório

O Panache oferece dois estilos: active record (métodos estáticos na própria entidade, como `TarefaEntidade.listAll()`) e repositório. O exemplo usa repositório, porque é ele que implementa a porta:

```
@ApplicationScoped
public class RepositorioPanache
        implements RepositorioDeTarefas, PanacheRepositoryBase<TarefaEntidade, Integer> {

    @Override
    public List<Tarefa> listar() {
        return listAll(Sort.by("id")).stream().map(TarefaEntidade::paraTarefa).toList();
    }

    @Override
    public Optional<Tarefa> buscar(int id) {
        return findByIdOptional(id).map(TarefaEntidade::paraTarefa);
    }

    // Escrita precisa de transação; leitura, não.
    @Override
    @Transactional
    public Tarefa adicionar(NovaTarefa nova) {
        TarefaEntidade entidade = new TarefaEntidade();
        entidade.titulo = nova.titulo();
        persist(entidade);
        return entidade.paraTarefa();
    }
}
```

- `PanacheRepositoryBase<TarefaEntidade, Integer>` traz `listAll`, `findByIdOptional`, `persist` e outros; o segundo parâmetro é o tipo do id.
- A porta passou a devolver `Optional<Tarefa>` em `buscar`, e o recurso transforma ausência em `404` com `orElseThrow(NotFoundException::new)`.
- O `RepositorioEmMemoria` perdeu o `@ApplicationScoped`. Com dois beans implementando a mesma interface, o CDI não saberia qual injetar (em 14/09, seção 8.7, o build falhava por dependência ambígua). Ele continua útil no teste unitário do recurso (seção 7.5).

Sobre threads: métodos de recurso que devolvem um valor comum (não `Uni`) rodam numa thread de trabalho no Quarkus REST, não no event loop (guia de 14/09, seção 9.4). Por isso o JDBC bloqueante do Hibernate é seguro aqui sem nenhuma anotação extra.

> Erro comum: gravar sem `@Transactional`. O `persist` falha e a requisição devolve `500`:
>
> ```
> jakarta.persistence.TransactionRequiredException: Transaction is not active,
> consider adding @Transactional to your method to automatically activate one.
> ```
>
> Leituras (`listAll`, `findByIdOptional`) funcionam sem a anotação; escritas não.

📖 Ref. Quarkus — Using Transactions: https://quarkus.io/guides/transaction

5.6 O banco como última barreira

Um achado de 14/09 era que o Quarkus aceitava `{}` e `{"titulo":null}` no `POST`, criando uma tarefa com título `null` na lista em memória. Com o banco, a coluna é `NOT NULL`, e o mesmo pedido agora falha:

```
ERROR: null value in column "titulo" of relation "tarefas" violates not-null constraint
org.hibernate.exception.ConstraintViolationException: could not execute statement
```

A resposta é `500 Internal Server Error`. O dado ruim não entrou, mas o cliente recebeu um erro do servidor por uma falha dele, que deveria ser `400` ou `422`. A restrição do banco é a última barreira; a primeira é validar a entrada, que é critério da rubrica. O Passo 11 do exemplo faz isso nos dois stacks (capítulo 10).

---

6. Exposed × Panache, lado a lado

	Ktor + Exposed	**Quarkus + Panache**
Estilo	SQL em Kotlin (DSL)	ORM: objetos mapeados para tabelas
Tabela no código	`object Tarefas : Table("tarefas")`	`@Entity class TarefaEntidade`
Consulta	`Tarefas.selectAll().where { ... }`	`findByIdOptional(id)`, `list("feita", true)`
Transação	`suspendTransaction(db) { ... }` explícito	`@Transactional` no método

	Ktor + Exposed	Quarkus + Panache
Pool	escolhido e configurado por você (HikariCP)	Agroal, pela extensão
Migração na subida	chamada a `Flyway.migrate()` no código	`quarkus.flyway.migrate-at-start=true`
Banco em desenvolvimento	`docker compose up -d`	Dev Services, automático
Banco nos testes	Testcontainers, declarado no teste	Dev Services, automático
Esquema	nunca `SchemaUtils.create`	`schema-management.strategy=none`
Bloqueio de thread	`suspend` + `suspendTransaction`	método comum roda em worker thread

Qual é melhor depende do grupo. Com Exposed, cada SQL é visível e nada acontece sem estar escrito. Com Panache, o CRUD sai quase pronto, mas vale conhecer o que o Hibernate faz por baixo: carregamento preguiçoso de relacionamentos, cache de primeiro nível dentro da transação e o problema N+1 (uma consulta para a lista e mais uma por item) aparecem assim que o domínio tiver relacionamentos.

📖 Ref. Quarkus — Using Hibernate ORM and Jakarta Persistence: https://quarkus.io/guides/hibernate-orm

---

7. Testes: da rota ao banco de verdade

7.1 Que testes escrever

A pirâmide de testes resume a ideia: muitos testes pequenos e rápidos na base, menos testes grandes e lentos no topo. Para a API de tarefas:

- Testes de rota com o repositório em memória: verificam HTTP (status, JSON, parâmetros) sem banco. Rodam em milissegundos, sem Docker.
- Testes de integração com banco de verdade: verificam o que só o banco mostra (a migração aplica, o SQL está certo, o id vem do `SERIAL`, a restrição `NOT NULL` vale).

A rubrica pede os dois: unitários do domínio e integração com Testcontainers, passando no Docker Desktop e no CI sem mudar configuração.

📖 Ref. Martin Fowler — Test Pyramid: https://martinfowler.com/bliki/TestPyramid.html

7.2 Ktor: rota sem banco

```
// Rotas com o repositório em memória: sem Docker, em milissegundos.
class RotasTest {

    @Test
    fun `id que nao e numero devolve 400`() = testApplication {
        application { configurar(RepositorioEmMemoria()) }
        assertEquals(HttpStatusCode.BadRequest, client.get("/tarefas/abc").status)
    }

    @Test
    fun `id existente devolve 200`() = testApplication {
        application { configurar(RepositorioEmMemoria()) }
        assertEquals(HttpStatusCode.OK, client.get("/tarefas/1").status)
    }
}
```

Os dois testes rodaram em 0,5 segundo. O que torna isso possível é `configurar` receber a porta: o teste passa a implementação em memória, e o resto da aplicação não percebe.

7.3 Testcontainers

Testcontainers é uma biblioteca que sobe containers Docker a partir do teste: um PostgreSQL de verdade, na versão que você escolher, descartado no fim. Isso resolve o "no meu computador funciona": o banco do teste é o mesmo em qualquer máquina com Docker, inclusive no CI.

```
testImplementation("org.testcontainers:testcontainers-postgresql:2.0.5")
```

No Testcontainers 2.x, o módulo se chama `testcontainers-postgresql` e a classe fica em `org.testcontainers.postgresql.PostgreSQLContainer`. Tutoriais com `org.testcontainers:postgresql` e `org.testcontainers.containers.PostgreSQLContainer` são da série 1.x.

📖 Ref. Testcontainers — Postgres Module: https://java.testcontainers.org/modules/databases/postgres/

7.4 Ktor: integração com PostgreSQL

```
class TarefasTest {

    companion object {
        // Um PostgreSQL de verdade, num container descartável, para a classe inteira.
        private val postgres = PostgreSQLContainer("postgres:17-alpine").apply { start() }
    }

    private fun ApplicationTestBuilder.appComBanco() = application {
        modulo(ConfigBanco(postgres.jdbcUrl, postgres.username, postgres.password))
    }

    @Test
    fun `lista as tarefas criadas pela migracao`() = testApplication {
        appComBanco()
        val cliente = createClient { install(ContentNegotiation) { json() } }
        val tarefas = cliente.get("/tarefas").body<List<Tarefa>>()
        assertTrue(tarefas.any { it.titulo == "Estudar Ktor" })
    }

    @Test
    fun `cria e depois busca pelo id`() = testApplication {
        appComBanco()
        val cliente = createClient { install(ContentNegotiation) { json() } }
        val resposta = cliente.post("/tarefas") {
            contentType(ContentType.Application.Json)
            setBody(NovaTarefa("Escrever testes"))
        }
        assertEquals(HttpStatusCode.Created, resposta.status)
        val criada = resposta.body<Tarefa>()
        assertEquals(criada, cliente.get("/tarefas/${criada.id}").body<Tarefa>())
    }

    @Test
    fun `id inexistente devolve 404`() = testApplication {
        appComBanco()
        assertEquals(HttpStatusCode.NotFound, client.get("/tarefas/9999").status)
    }
}
```

- O container fica no `companion object`: sobe uma vez para a classe, não uma vez por teste. Os quatro testes da classe (os três acima e o de OpenAPI, seção 8.2) rodaram em 2,6 segundos com a imagem já baixada.
- Cada `testApplication` chama `modulo`, que roda o Flyway. Na primeira vez ele migra; nas seguintes, registra `Schema "public" is up to date`.
- `createClient { install(ContentNegotiation) { json() } }` cria um cliente que converte JSON em objetos, e `body<Tarefa>()` faz a conversão. Precisa da dependência `io.ktor:ktor-client-content-negotiation` no escopo de teste.
- `useJUnitPlatform()` no `build.gradle.kts`: o `kotlin("test")` escolhe o JUnit 5 como executor quando a task de teste usa a plataforma JUnit.

Os testes compartilham o banco da classe. A tarefa criada em um teste aparece para os outros, por isso as asserções usam `any { ... }` em vez de contar itens. Quando o isolamento importar, limpe as tabelas antes de cada teste ou use um banco por classe.

> Erro comum: logs de DEBUG do Testcontainers inundando a saída do teste. Sem configuração de log, o Logback mostra tudo. O exemplo tem `src/test/resources/logback-test.xml` com o nível `WARN` e `INFO` só para o Flyway e para os containers.

📖 Ref. Ktor — Testing: https://ktor.io/docs/server-testing.html

7.5 Quarkus: `@QuarkusTest` com Dev Services

```
// @QuarkusTest sobe a aplicação inteira; sem URL de banco no perfil de teste,
// o Dev Services sobe um PostgreSQL no Docker e o Flyway aplica as migrações.
@QuarkusTest
class RecursoDeTarefasTest {

    @Test
    void listaAsTarefasCriadasPelaMigracao() {
        given().when().get("/tarefas")
                .then().statusCode(200)
                .body("titulo", hasItem("Estudar Quarkus"));
    }

    @Test
    void criaEDepoisBuscaPeloId() {
        int id = given().contentType(ContentType.JSON).body("{\"titulo\":\"Escrever
        testes\"}")
                .when().post("/tarefas")
                .then().statusCode(201)
                .extract().path("id");

        given().when().get("/tarefas/" + id)
                .then().statusCode(200)
                .body("titulo", equalTo("Escrever testes"));
    }

    @Test
    void idInexistenteDevolve404() {
        given().when().get("/tarefas/9999").then().statusCode(404);
    }
}
```

- Não há nenhuma linha sobre banco no teste. O Dev Services usa o Testcontainers por baixo.
- O equivalente ao teste de rota sem banco do Ktor é um teste JUnit comum, sem `@QuarkusTest`: `RecursoDeTarefasUnitarioTest` cria `new RecursoDeTarefas()`, põe um `RepositorioEmMemoria` no campo `repositorio` (o que o CDI faria) e chama os métodos direto. Os três testes rodaram em 0,017 segundo, sem Docker.
- `quarkus.http.test-port=8083` no `application.properties` resolve o achado de 14/09: a porta padrão de teste (8081) colidia com a porta da aplicação, e `mvn test` falhava com o modo dev aberto.

> Erro comum: Docker Desktop atualizado e `@QuarkusTest` falhando com `Could not find a valid Docker environment`, com `BadRequestException (Status 400)` nos detalhes. O Quarkus 3.28 traz o Testcontainers 1.21.3, que não conversa com o Docker Engine 29. A correção no exemplo é importar o BOM do Testcontainers 1.21.4 antes do BOM do Quarkus, no `dependencyManagement`:
>
> ```
> <dependency>
>   <groupId>org.testcontainers</groupId>
>   <artifactId>testcontainers-bom</artifactId>
>   <version>1.21.4</version>
>   <type>pom</type>
>   <scope>import</scope>
> </dependency>
> ```
>
> Versões mais novas do Quarkus (3.33 em diante) já trazem o Testcontainers 2.x.

No fim de cada `mvn test` com Java 25, aparece um `IllegalAccessError` sobre `thread-local-reset` e `--add-opens`. Ele acontece no encerramento, depois de os testes já terem passado, e não afeta o resultado.

📖 Ref. Quarkus — Testing your application: https://quarkus.io/guides/getting-started-testing

7.6 kotlin-test × JUnit 5

	Ktor (kotlin-test)	**Quarkus (JUnit 5)**
Anotação	`@Test` de `kotlin.test`	`@Test` de `org.junit.jupiter.api`
Asserção	`assertEquals`, `assertTrue`	Hamcrest via REST Assured (`equalTo`, `hasItem`)
Sobe a aplicação	`testApplication { }`, em memória, sem porta	`@QuarkusTest`, com porta real (8083)
Banco	Testcontainers, declarado no teste	Dev Services, implícito
Executor	JUnit Platform (`useJUnitPlatform()`)	JUnit Platform (Surefire)

O `kotlin-test` é uma camada fina: no Gradle com JUnit Platform, os testes rodam no JUnit 5. Dá para misturar os dois no mesmo projeto Kotlin.

---

8. OpenAPI: a documentação que sai do código

8.1 O que é

OpenAPI é um formato (YAML ou JSON) que descreve uma API HTTP: caminhos, métodos, parâmetros, corpos e respostas com seus esquemas. A partir dele saem a Swagger UI (uma página para explorar e testar a API) e clientes gerados.

Gerar a especificação do código evita a defasagem: se a rota mudar, a documentação muda junto.

📖 Ref. OpenAPI Specification 3.1.1: https://spec.openapis.org/oas/v3.1.1.html

8.2 Ktor: `describe` e Swagger UI

Dependências:

```
implementation("io.ktor:ktor-server-routing-openapi:3.5.2")
implementation("io.ktor:ktor-server-swagger:3.5.2")
```

Cada rota ganha um `.describe { }`:

```kotlin
@OptIn(ExperimentalKtorApi::class)
fun Application.rotas() {
    val repositorio by inject<RepositorioDeTarefas>()

    routing {
        route("/tarefas") {
            get { call.respond(repositorio.listar()) }.describe {
                summary = "Lista as tarefas"
                responses { HttpStatusCode.OK { schema = jsonSchema<List<Tarefa>>() } }
            }

            get("/{id}") {
                val id = call.parameters["id"]?.toIntOrNull()
                    ?: return@get call.respond(HttpStatusCode.BadRequest)
                val tarefa = repositorio.buscar(id)
                    ?: return@get call.respond(HttpStatusCode.NotFound)
                call.respond(tarefa)
            }.describe {
                summary = "Busca uma tarefa pelo id"
                parameters { path("id") { schema = jsonSchema<Int>() } }
                responses {
                    HttpStatusCode.OK { schema = jsonSchema<Tarefa>() }
                    HttpStatusCode.NotFound { description = "Tarefa inexistente" }
                }
            }

            post {
                val nova = call.receive<NovaTarefa>()
                call.respond(HttpStatusCode.Created, repositorio.adicionar(nova))
            }.describe {
                summary = "Cria uma tarefa"
                requestBody { schema = jsonSchema<NovaTarefa>() }
                responses { HttpStatusCode.Created { schema = jsonSchema<Tarefa>() } }
            }
        }

        swaggerUI(path = "docs") {
            info = OpenApiInfo(title = "Tarefas", version = "1.0")
        }
```

```
    }
}
```

- `swaggerUI(path = "docs")` serve a página em `/docs` e a especificação, gerada das rotas, em `/docs/documentation.yaml`.
- `jsonSchema<Tarefa>()` deriva o esquema da classe `@Serializable`. No resultado, `feita` aparece como opcional, porque tem valor padrão na `data class`.
- A API de `describe` é experimental no Ktor 3.5 (`@OptIn(ExperimentalKtorApi::class)`): pode mudar entre versões.

Trecho da especificação gerada (OpenAPI 3.1.1):

```yaml
paths:
  /tarefas/{id}:
    get:
      summary: Busca uma tarefa pelo id
      parameters:
      - name: id
        in: path
        required: true
        schema:
          type: integer
      responses:
        "200":
          content:
            application/json:
              schema:
                $ref: "#/components/schemas/Tarefa"
        "404":
          description: Tarefa inexistente
```

> Erro comum (Ktor): parâmetro de caminho documentado como `string`. Sem a linha `parameters { path("id") { ... } }`, o `{id}` aparece com `type: string`, porque na rota ele chega como texto. Quem gera cliente a partir dessa especificação recebe o tipo errado. Declare o tipo real.

Há uma alternativa mantida pela comunidade, `io.github.smiley4:ktor-openapi` (o "ktor-openapi" do plano de curso), em que a documentação vai como primeiro argumento de `get` / `post`. Os dois produzem OpenAPI a partir do código, e o grupo escolhe um. O MUSI usava o smiley4 e passou para o gerador nativo, mantido pela própria JetBrains: lá, os blocos `describe` de todas as rotas ficam num

arquivo à parte (`RotasDoc.kt`), o que mantém a árvore de rotas legível sem perder a geração a partir do código.

📖 Ref. Ktor — OpenAPI specification generation: https://ktor.io/docs/openapi-spec-generation.html

📖 Ref. Ktor — Swagger UI: https://ktor.io/docs/server-swagger-ui.html

📖 Ref. ktor-openapi-tools (smiley4): https://github.com/SMILEY4/ktor-openapi-tools

8.3 Quarkus: smallrye-openapi

```
<dependency>
  <groupId>io.quarkus</groupId>
  <artifactId>quarkus-smallrye-openapi</artifactId>
</dependency>
```

Só com a dependência, o Quarkus lê as anotações JAX-RS e gera a especificação em `/q/openapi` . O parâmetro `{id}` sai como `integer` / `int32` , porque o método recebe `@PathParam("id") int id` . Para os resumos, o exemplo usa `@Operation` :

```
@GET
@Path("/{id}")
@Operation(summary = "Busca uma tarefa pelo id")
public Tarefa buscar(@PathParam("id") int id) {
    return repositorio.buscar(id).orElseThrow(NotFoundException::new);
}
```

- A Swagger UI fica em `/q/swagger-ui` , mas por padrão só nos modos dev e test. No `java -jar` (produção), `/q/openapi` respondeu `200` e `/q/swagger-ui/` respondeu `404` . Para incluir a UI no build de produção: `quarkus.swagger-ui.always-include=true` (conferido: com ela, `/q/swagger-ui/` respondeu `200`).
- `quarkus.smallrye-openapi.info-title=Tarefas` define o título da especificação.

📖 Ref. Quarkus — Using OpenAPI and Swagger UI: https://quarkus.io/guides/openapi-swaggerui

8.4 Comparação

	Ktor	Quarkus
De onde vem a especificação	`.describe { }` em cada rota	anotações JAX-RS, automáticas

	Ktor	Quarkus
Tipos	`jsonSchema<T>()`, via kotlinx.serialization	reflexão sobre os tipos Java
Especificação	`/docs/documentation.yaml`	`/q/openapi`
Swagger UI	`/docs`	`/q/swagger-ui` (dev e test; em prod, com `always-include`)
Estabilidade	experimental no Ktor 3.5	estável

No Quarkus o esforço é menor: sem escrever nada, a especificação já existe. No Ktor, como as rotas são código e não anotações, a descrição também é código.

---

9. Rodar tudo junto

9.1 Banco com Docker Compose

O exemplo do Ktor traz um `compose.yaml` só com o banco:

```yaml
services:
  banco:
    image: postgres:17-alpine
    environment:
      POSTGRES_DB: tarefas
      POSTGRES_USER: tarefas
      POSTGRES_PASSWORD: tarefas
    ports: ["5432:5432"]
    volumes: [dados:/var/lib/postgresql/data]
volumes:
  dados:
```

- `volumes`: os dados ficam num volume nomeado e sobrevivem a `docker compose down`. `docker compose down -v` apaga o volume e volta ao banco vazio.
- Os valores de ambiente batem com os padrões de `ConfigBanco.doAmbiente()` e do perfil `%prod` do Quarkus, então as duas APIs usam esse banco sem configuração extra.

Na Sprint 1, a rubrica pede o banco subindo por `docker compose`. Na Sprint 2, o mesmo arquivo cresce: `docker compose up` passa a subir a API, o serviço Go e o banco, do zero. A API entra quando houver um `Dockerfile` (como os do MUSI).

📖 Ref. Docker Compose: https://docs.docker.com/compose/

9.2 Sequência

```
docker compose up -d
```

```
./gradlew run
```

```
curl -s localhost:8080/tarefas
curl -s -i -X POST localhost:8080/tarefas -H 'Content-Type: application/json' -d
        '{"titulo":"Persistir"}'
```

Respostas conferidas:

```
[{"id":1,"titulo":"Estudar Ktor","feita":false},{"id":2,"titulo":"Estudar
Quarkus","feita":false}]
HTTP/1.1 201 Created
{"id":3,"titulo":"Persistir","feita":false}
```

Depois de parar e subir a aplicação de novo, `GET /tarefas` devolveu as três tarefas, e o Flyway registrou `Schema "public" is up to date`.

Para o Quarkus, contra o mesmo banco:

```
mvn package -DskipTests
java -jar target/quarkus-app/quarkus-run.jar
```

A aplicação subiu em 1,4 segundo na porta 8081. Como o banco era novo, o Flyway do Quarkus aplicou o `V1`. Se o Ktor já tivesse migrado o banco, o Quarkus encontraria o histórico e não faria nada: os dois exemplos compartilham a mesma migração.

9.3 Respostas que diferem entre os stacks

Pedido	Ktor	Quarkus
`GET /tarefas/abc`	`400` (o código testa `toIntOrNull()`)	`404` (nenhum método aceita `abc` como `int`)
`GET /tarefas/999`	`404`	`404`

Pedido	Ktor	Quarkus
`POST` com `{}`	`400` , `Failed to convert request body to class ...NovaTarefa` (kotlinx.serialization exige `titulo`)	`500` (chega ao banco e viola `NOT NULL`)

As duas primeiras linhas são escolhas defensáveis. A terceira mostra o que acontece sem validação (seção 5.6); o capítulo 10 a resolve.

9.4 No MUSI: o mesmo assunto, em escala de projeto

O exemplo desta aula tem uma entidade e um repositório. O projeto de referência (`github.com/fmarquesfilho/musi`) faz o que a rubrica da Sprint 1 pede, nos dois stacks, e serve para ver as mesmas ideias com mais peças:

Para ver	Onde, no MUSI
Duas entidades com relacionamento	`Obra` 1:N `Anotação` , com chave estrangeira e `ON DELETE CASCADE` , na migração `V1`
Rota aninhada	`/obras/{id}/anotacoes/{anotacaoId}`
Paginação e filtros no SQL	`?pagina=&tamanho=&ordem=&artista=&anoDe=&anoAte=&dimensao=&valor=` , com teto de 100
A mesma migração em dois stacks	pastas `db/migration` iguais em `api-ktor` e `api-quarkus` , comparadas pelo CI
Teste de arquitetura	`ArquiteturaTest` , com ArchUnit, nos dois stacks
Decisão registrada	`docs/decisoes/0004-persistencia-postgresql-flyway.md`

Quatro cuidados que apareceram ao montar isso, e que valem para o projeto de vocês:

> Erro comum (Quarkus): a aplicação não sobe sem banco. Sem URL, o Quarkus desativa o datasource sozinho — mas o Hibernate e o Flyway continuam exigindo um, e a subida falha com `Unable to find datasource '<default>'` . Para subir sem banco (o caso de um deploy antes de ter Postgres), é preciso desligar os dois: `quarkus.hibernate-orm.active=false` e `quarkus.flyway.active=false` .

> Erro comum (Quarkus): parâmetro de consulta com tipo errado vira `404` . `?pagina=abc` em `@QueryParam("pagina") Integer pagina` não dá `400` : a especificação Jakarta REST manda devolver `404` . É a mesma regra da tabela da seção 9.3, agora na query string. Para responder `400` , receba como `String` e converta à mão.

> Erro comum (Exposed 1.5): `uuid()` não é `java.util.UUID`. Os pacotes agora são `org.jetbrains.exposed.v1.core` e `.v1.jdbc`, e `uuid()` mapeia `kotlin.uuid.Uuid`. Para a `java.util.UUID` de sempre, a coluna é `javaUUID()`.

> Erro comum (Hibernate): trocar uma lista inteira num `PUT`. Com `@ElementCollection` e `@OrderColumn`, substituir a lista pode violar a chave da tabela filha quando os itens só trocam de posição. Esvaziar a coleção e dar `flush()` antes de repor resolve.

O MUSI também mostra como rodar a suíte sem Docker: os testes de integração levam a tag `integracao`, e `./gradlew test -PsemDocker` (ou `mvn test -DexcludedGroups=integracao`) deixa só os que não precisam de banco. No CI, tudo roda.

---

10. Validação e erros em *problem details*

É o Passo 11 dos exemplos. A rubrica pede validação em todas as entradas, erros no formato *problem details* e nenhuma resposta `500` para erro do cliente.

10.1 400 ou 422

O que está errado	Status	Exemplo
A forma: JSON quebrado, campo faltando, tipo errado, id que não é número	`400`	`{"titulo":[1]}`
A regra: a forma está certa, o conteúdo não	`422`	`{"titulo":" "}`
O recurso não existe	`404`	`GET /tarefas/9999`
Erro do servidor	`500`, sem detalhe interno	exceção não prevista

O capítulo 6 da leitura de 14/09 explica a RFC 9457 e a escolha entre `400` e `422`. Aqui está como o exemplo aplica.

10.2 A regra fica no domínio

A regra não sabe de HTTP. Ela devolve a lista do que está errado, e quem chama decide o que fazer:

```kotlin
// Tarefa.kt
@Serializable
data class NovaTarefa(val titulo: String) {
    fun violacoes(): List<String> = when {
        titulo.isBlank() -> listOf("titulo: não pode ficar em branco")
        titulo.length > 200 -> listOf("titulo: no máximo 200 caracteres")
        else -> emptyList()
    }
}

class EntradaInvalida(val violacoes: List<String>) :
        RuntimeException(violacoes.joinToString("; "))
```

```java
// NovaTarefa.java
public record NovaTarefa(String titulo) {
    public List<String> violacoes() {
        if (titulo == null || titulo.isBlank()) {
            return List.of("titulo: não pode ficar em branco");
        }
        if (titulo.length() > 200) {
            return List.of("titulo: no máximo 200 caracteres");
        }
        return List.of();
    }
}
```

No Java, `null` também conta: com Jackson, o corpo `{}` chega como título nulo.

A rota e o recurso só lançam a exceção:

```kotlin
post {
    val nova = call.receive<NovaTarefa>()
    val violacoes = nova.violacoes()
    if (violacoes.isNotEmpty()) throw EntradaInvalida(violacoes)
    val criada = repositorio.adicionar(nova)
    call.response.header(HttpHeaders.Location, "/tarefas/${criada.id}")
    call.respond(HttpStatusCode.Created, criada)
}
```

```java
@POST
public Response criar(NovaTarefa nova) {
    if (nova == null) {
        throw new BadRequestException("O corpo da requisição é obrigatório");
    }
    List<String> violacoes = nova.violacoes();
    if (!violacoes.isEmpty()) {
        throw new EntradaInvalida(violacoes);
    }
    Tarefa criada = repositorio.adicionar(nova);
    return Response.created(URI.create("/tarefas/" + criada.id())).entity(criada).build();
}
```

O `201` passa a levar o cabeçalho `Location` com o endereço da tarefa criada, como pede a tarefa T5.

10.3 Um lugar só decide a resposta

No Ktor, o plugin `StatusPages` (dependência `io.ktor:ktor-server-status-pages`) associa cada exceção a uma resposta:

```
fun Application.tratarErros() {
    install(StatusPages) {
        exception<BadRequestException> { call, causa ->
            call.responderProblema(
                HttpStatusCode.BadRequest, "requisicao-malformada", "Requisição malformada",
                causa.cause?.message ?: causa.message,
            )
        }
        exception<NotFoundException> { call, causa ->
            call.responderProblema(HttpStatusCode.NotFound, "nao-encontrado", "Recurso
        inexistente", causa.message)
        }
        exception<EntradaInvalida> { call, causa ->
            call.responderProblema(
                HttpStatusCode.UnprocessableEntity, "entrada-invalida", "Entrada inválida",
                "A entrada viola ${causa.violacoes.size} regra(s).", causa.violacoes,
            )
        }
        exception<Throwable> { call, causa ->
            call.application.environment.log.error("erro não tratado", causa)
            call.responderProblema(HttpStatusCode.InternalServerError, "interno", "Erro
        interno")
        }
    }
}
```

No Quarkus REST, o equivalente é uma classe com um método `@ServerExceptionMapper` por tipo de exceção:

```java
public class Erros {

    @ServerExceptionMapper
    public Response entradaInvalida(EntradaInvalida e) {
        return problema(422, "entrada-invalida", "Entrada inválida",
                "A entrada viola " + e.violacoes().size() + " regra(s).", e.violacoes());
    }

    @ServerExceptionMapper
    public Response jsonInvalido(MismatchedInputException e) { /* 400 */ }

    @ServerExceptionMapper
    public Response web(WebApplicationException e) { /* 400, 404 e os demais do Jakarta REST */ }

    @ServerExceptionMapper
    public Response inesperado(RuntimeException e) { /* 500, sem detalhe interno */ }
}
```

Nos dois, a resposta sai com o tipo `application/problem+json`. Conferido:

```
curl -s -i -X POST localhost:8080/tarefas -H 'Content-Type: application/json' -d '{"titulo":"   "}'
```

```
HTTP/1.1 422 Unprocessable Entity
Content-Type: application/problem+json

{"type":"/problemas/entrada-invalida","title":"Entrada inválida","status":422,"detail":"A entrada viola 1 regra(s).","violacoes":["titulo: não pode ficar em branco"]}
```

`type` identifica a classe do erro; `detail`, esta ocorrência. `violacoes` é um membro de extensão, que a RFC permite.

10.4 As respostas depois do Passo 11

Pedido	Ktor	Quarkus
`POST` com `{"titulo":"  "}`	`422`	`422`
`POST` com `{}`	`400`: a kotlinx.serialization exige o campo	`422`: o Jackson cria o título nulo, e a regra recusa

Pedido	Ktor	Quarkus
`POST` sem corpo	`400`	`400`
`GET /tarefas/abc`	`400`	`404`
`GET /tarefas/999`	`404`	`404`
`POST` válido	`201` com `Location: /tarefas/{id}`	`201` com `Location` (a URL completa)

O `500` da seção 5.6 deixa de acontecer: a entrada inválida não chega ao banco.

No Quarkus, a alternativa comum é o Bean Validation (`quarkus-hibernate-validator` , `@NotBlank` no record e `@Valid` no parâmetro). O exemplo usa uma função do domínio, igual à do Ktor, para o domínio não depender de anotações de framework (capítulo 11). O MUSI usa as duas formas: Bean Validation para a forma do corpo e regras do domínio para o conteúdo.

> Erro comum (Quarkus): campo com tipo errado respondido fora do formato. Sem um método para `MismatchedInputException` , o Quarkus responde `{"titulo":[1,2]}` com um JSON próprio (`{"objectName":"Class","attributeName":"titulo",...}`), e não com *problem details*.

📖 Ref. RFC 9457 — Problem Details for HTTP APIs: https://www.rfc-editor.org/rfc/rfc9457

📖 Ref. Ktor — Status pages: https://ktor.io/docs/server-status-pages.html

📖 Ref. Quarkus — REST, exception mapping: https://quarkus.io/guides/rest#exception-mapping

---

11. Camadas, teste de arquitetura e o CRUD completo

11.1 Os pacotes

É o Passo 12 dos exemplos. Até aqui, tudo ficava num pacote só. Agora o código se divide:

```
br.ufrn.exemplo.tarefas
├── dominio/          Tarefa, NovaTarefa, EntradaInvalida, RepositorioDeTarefas (a porta)
└── adaptadores/
    ├── http/         rotas ou recurso, erros
    ├── banco/        Exposed ou Panache
    └── memoria/      RepositorioEmMemoria
```

A regra de dependência do capítulo 7 da leitura de 14/09 vira uma regra sobre pacotes: os adaptadores dependem do domínio, e o domínio não depende de ninguém. No Ktor, `Aplicacao.kt` fica na raiz e liga as camadas; no Quarkus, quem liga é o CDI.

O projeto do grupo tem mais uma camada, a dos casos de uso (o MUSI a chama de `aplicacao`). O exemplo, com três operações, não precisa dela.

11.2 O teste que verifica a regra

Pacote é convenção: nada impede um `import` de framework no domínio. O ArchUnit lê as classes compiladas e transforma a regra num teste (dependência de teste `com.tngtech.archunit:archunit:1.5.0`, nos dois stacks):

```
class ArquiteturaTest {


    private val classes = ClassFileImporter()
        .withImportOption(ImportOption.DoNotIncludeTests())
        .importPackages("br.ufrn.exemplo.tarefas")


    @Test
    fun `o dominio nao conhece framework, banco nem adaptadores`() =
        regraDoDominio.check(classes)


    companion object {
        val regraDoDominio: ArchRule = noClasses().that().resideInAPackage("..dominio..")
            .should().dependOnClassesThat().resideInAnyPackage(
                "io.ktor..", "org.koin..", "org.jetbrains.exposed..", "java.sql..",
        "javax.sql..",
                "com.zaxxer..", "org.flywaydb..", "..adaptadores..",
            )
    }
}
```

No Quarkus, a lista proibida é `jakarta..`, `io.quarkus..`, `org.hibernate..`, `com.fasterxml..`, `java.sql..` e `..adaptadores..`.

A rubrica pede que o teste falhe quando a regra é violada. Conferido: com um método que devolve `jakarta.ws.rs.core.Response` no `dominio/Tarefa.java`, `mvn test` falha com

```
Architecture Violation [Priority: MEDIUM] - Rule 'no classes that reside in a package
'..dominio..' should depend on classes that reside in any package ['jakarta..', ...]'
was violated (3 times)
```

O exemplo tem também um teste que faz essa prova sozinho: `a regra do dominio pega uma violacao` avalia a regra contra uma classe que só existe nos testes (`arquitetura/violacao/dominio/Contaminado`) e confere que há violação.

No Ktor, o domínio do exemplo aceita `kotlinx.serialization` (o `@Serializable` da `Tarefa`). No MUSI, o domínio é Kotlin puro e a serialização fica em DTOs do adaptador web.

No Kotlin, o Konsist é uma alternativa ao ArchUnit: lê o código-fonte em vez das classes compiladas.

> Erro comum (Quarkus): `GET /tarefas is declared by` duas classes, ao abrir o `mvn quarkus:dev` depois de mudar classes de pacote. São arquivos `.class` antigos na pasta `target/`. `mvn clean` resolve.

📖 Ref. ArchUnit — User Guide: https://www.archunit.org/userguide/html/000_Index.html

📖 Ref. Konsist: https://docs.konsist.lemonappdev.com/

11.3 O que o projeto precisa além do exemplo

O exemplo tem uma entidade e três operações. A rubrica pede duas entidades com relacionamento, todas as operações, paginação e filtros. O MUSI mostra essas peças; os trechos abaixo são dele.

Relacionamento. Na migração, a chave estrangeira:

```
CREATE TABLE anotacoes (
    id        BIGINT       GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    obra_id   VARCHAR(64)  NOT NULL REFERENCES obras (id) ON DELETE CASCADE,
    -- ...
);

-- A chave estrangeira não cria índice sozinha no PostgreSQL; a rota aninhada consulta por
       ela.
CREATE INDEX anotacoes_obra ON anotacoes (obra_id);
```

No Exposed, a coluna aponta para a outra tabela; no Panache, a entidade tem um `@ManyToOne`:

```
val obraId = varchar("obra_id", 64).references(Obras.id, onDelete = ReferenceOption.CASCADE)
```

```
@ManyToOne(fetch = FetchType.LAZY, optional = false)
@JoinColumn(name = "obra_id")
public ObraEntidade obra;
```

Na API, o relacionamento aparece como rota aninhada: `/obras/{id}/anotacoes`.

Paginação e filtro, no SQL. O banco devolve só a fatia pedida; a aplicação não carrega a tabela inteira:

```
val itens = Anotacoes.selectAll().where(onde)
    .orderBy(Anotacoes.criadoEm to SortOrder.ASC, Anotacoes.id to SortOrder.ASC)
    .limit(pedido.tamanho).offset(pedido.deslocamento)
    .map { it.paraAnotacao() }
```

```
var itens = find(onde + " order by criadoEm, id", params)
    .page(Page.of(pedido.pagina(), pedido.tamanho()))
    .list().stream().map(AnotacaoEntidade::paraAnotacao).toList();
```

- O tamanho da página tem um teto (100, no MUSI): sem ele, `?tamanho=1000000` traria a tabela inteira.
- A listagem tem ordem definida (`ORDER BY`): sem ela, as páginas podem repetir ou pular itens.
- A resposta traz também o total, para quem consome saber quantas páginas há.

Atualizar e remover seguem os métodos e status do capítulo 6 da leitura de 14/09: `PUT` ou `PATCH` com `200`, `DELETE` com `204`, `404` para id inexistente.

---

12. Exercícios e dúvidas frequentes

12.1 Perguntas de fixação

1. Por que as rotas não mudaram quando o repositório passou a usar PostgreSQL?
2. O que o Flyway guarda na tabela `flyway_schema_history`, e para que serve o checksum?
3. Por que a rubrica proíbe `ddl-auto` e `SchemaUtils.create`?
4. O que acontece com uma consulta do Exposed fora de `transaction` ou `suspendTransaction`?
5. Por que a interface do Ktor ganhou `suspend` e a do Quarkus não?
6. Qual a diferença entre `RepositorioEmMemoria` e `RepositorioPostgres` do ponto de vista dos testes?
7. O que o Dev Services faz, e em quais perfis ele entra?
8. Antes do Passo 11, por que o `POST {}` no Quarkus dava `500`? Onde isso foi resolvido?
9. Qual a diferença entre responder `400` e `422`?
10. Por que a regra de validação fica no domínio, e não na rota ou no recurso?
11. O que o teste de arquitetura verifica, e como saber que ele falha quando deve?

12.2 Exercícios práticos

1. Adicione a coluna `prazo DATE` (opcional) numa migração `V2__adiciona_prazo.sql`. Atualize a tabela do Exposed ou a entidade, o domínio e as respostas. Confira no log que só o `V2` foi aplicado.
2. Implemente `PATCH /tarefas/{id}/feita`, que marca a tarefa como feita, nos dois stacks. Escreva um teste de integração que cria, marca e confere.
3. Edite o `V1` depois de aplicado e veja a mensagem de checksum. Desfaça a edição.
4. No Quarkus, troque a função `violacoes()` por Bean Validation (`quarkus-hibernate-validator`, `@NotBlank` no `titulo` e `@Valid` no parâmetro). Compare a resposta de `POST {}` e veja o que acontece com o teste de arquitetura.
5. No Ktor, escreva um teste que roda com o banco vazio: suba um segundo container, rode só o Flyway e confirme que existem exatamente duas tarefas.
6. Remova `parameters { path("id") ... }` e compare o tipo do `{id}` no `documentation.yaml`.
7. Acrescente uma segunda regra à `NovaTarefa` (por exemplo, título com pelo menos 3 caracteres) e um teste de rota que confira o `422` com as duas violações possíveis.
8. Importe uma classe do framework no pacote `dominio` e rode os testes. Leia a mensagem do ArchUnit e desfaça.
9. Implemente `GET /tarefas?feita=true&pagina=0&tamanho=10`, com a paginação no SQL e um teto para `tamanho`.

12.3 Dúvidas frequentes

- Preciso de Docker para rodar os testes? Para os de integração, sim, nos dois stacks. Os testes de rota do Ktor com o repositório em memória e o teste unitário do recurso no Quarkus rodam sem Docker.
- Não tenho Docker, ou meu computador não dá conta. Como rodo? Num Codespace do repositório (Code → Codespaces), que é também o ambiente usado no laboratório. Ele traz Java 25, Maven, Go e Docker, com as dependências e as imagens do PostgreSQL já baixadas; a criação leva cerca de 5 minutos. Os comandos são os mesmos (`./gradlew run`, `mvn quarkus:dev`, `docker compose up -d`, `mise run test`), e as portas 8080 e 8081 aparecem na aba Portas. O uso gratuito mensal é limitado: pare o Codespace ao terminar.
- Posso usar o Codespace no projeto do grupo? Sim. Copie a pasta `.devcontainer/` para o repositório do grupo e ajuste o que for preciso; a feature `docker-in-docker` é o que permite rodar o `docker compose` e o Testcontainers lá dentro.
- Posso usar H2 em memória nos testes em vez de PostgreSQL? Funciona para coisas simples, mas o H2 não é o PostgreSQL: tipos, funções e mensagens de erro diferem. A rubrica pede Testcontainers, justamente para testar contra o banco real.
- A imagem do banco é `postgres:17-alpine` no Ktor e `postgres:17` no Quarkus. Faz diferença? Não para o exemplo. A do Dev Services é a padrão do Quarkus; dá para trocar com

`quarkus.datasource.devservices.image-name`.

- Os testes estão lentos na primeira vez. É normal? Sim. A primeira execução baixa as imagens (a `postgres:17` tem 158 MB). Depois, o container sobe em cerca de 1,5 segundo.
- Onde ficam os dados do banco do Compose? No volume `dados`. Para começar do zero: `docker compose down -v`.
- Exposed ou Ktorm? O plano de curso permite os dois. O exemplo usa Exposed por ser mantido pela JetBrains e ter integração documentada com Ktor.

Referências

Banco e JDBC

- PostgreSQL — tipos numéricos e `SERIAL`: https://www.postgresql.org/docs/17/datatype-numeric.html
- PostgreSQL — isolamento de transações: https://www.postgresql.org/docs/current/transaction-iso.html
- HikariCP: https://github.com/brettwooldridge/HikariCP

Migrações

- Flyway — Migrations: https://documentation.red-gate.com/flyway/flyway-concepts/migrations
- Flyway — Versioned migrations: https://documentation.red-gate.com/flyway/flyway-concepts/migrations/versioned-migrations
- Quarkus — Using Flyway: https://quarkus.io/guides/flyway

Ktor e Exposed

- Exposed — Get started: https://www.jetbrains.com/help/exposed/get-started-with-exposed.html
- Exposed — CRUD operations (DSL): https://www.jetbrains.com/help/exposed/dsl-crud-operations.html
- Exposed — Transactions: https://www.jetbrains.com/help/exposed/transactions.html
- Exposed — Working with DataSource: https://www.jetbrains.com/help/exposed/working-with-datasource.html
- Exposed — Migrations: https://www.jetbrains.com/help/exposed/migrations.html
- Ktor — Testing: https://ktor.io/docs/server-testing.html
- Ktor — OpenAPI specification generation: https://ktor.io/docs/openapi-spec-generation.html
- Ktor — OpenAPI: https://ktor.io/docs/server-openapi.html
- Ktor — Swagger UI: https://ktor.io/docs/server-swagger-ui.html
- ktor-openapi-tools (smiley4), a alternativa da comunidade: https://github.com/SMILEY4/ktor-openapi-tools

Quarkus

- Simplified Hibernate ORM with Panache: https://quarkus.io/guides/hibernate-orm-panache
- Using Hibernate ORM and Jakarta Persistence: https://quarkus.io/guides/hibernate-orm
- Using Transactions: https://quarkus.io/guides/transaction
- Dev Services for Databases: https://quarkus.io/guides/databases-dev-services
- Testing your application: https://quarkus.io/guides/getting-started-testing
- Using OpenAPI and Swagger UI: https://quarkus.io/guides/openapi-swaggerui

Testes

- Testcontainers for Java: https://java.testcontainers.org/
- Testcontainers — Postgres Module: https://java.testcontainers.org/modules/databases/postgres/
- Testcontainers Guides: https://testcontainers.com/guides/
- Martin Fowler — Test Pyramid: https://martinfowler.com/bliki/TestPyramid.html

OpenAPI e ambiente

- OpenAPI Specification 3.1.1: https://spec.openapis.org/oas/v3.1.1.html
- Docker Compose: https://docs.docker.com/compose/